

SLIDING WINDOW Pattern

Complete DSA Notes — Concept, Identification Flowchart, Worked Example & C++ Code

DSA Pattern Series — Pattern #2

Prerequisite: Two Pointer Pattern

Compiled from video transcript + added explanations, tables & code

Table of Contents

1. What Is the Sliding Window Pattern?

Break the name into two parts and the whole pattern becomes obvious:

- **Window** → a contiguous chunk/range of an array or string that you are currently "looking at".
- **Sliding** → this window can grow bigger (expand) or shrink smaller on its own, and it moves forward as you process the array.

Simple Analogy

Think of looking through a physical window frame as you walk along a wall. At any moment you only see a small portion of the wall (the "window"). As you walk forward, the frame slides — and depending on how you adjust it, the frame can get wider (expand) or narrower (shrink). Sliding Window in DSA works exactly the same way over an array.

Built on Top of Two Pointers

Sliding Window is NOT a brand-new idea — it is Two Pointers with an extra layer of logic on top. That is exactly why Two Pointers is taught first.

- Two boundary variables are used: **low** (left edge) and **high** (right edge).
- Both pointers move in the SAME direction (left to right) — never backward.
- Moving **high++** while keeping low fixed → window **EXPANDS** (grows on the right side).
- Moving **low++** while keeping high fixed → window **SHRINKS** (cuts off from the left side).

Key Insight

The window only ever grows from the right end and only ever shrinks from the left end. That one rule is what makes the technique efficient — you never have to look at the whole window again from scratch.

2. Where Does Sliding Window Apply? (Checkpoint #1)

Before even thinking about Sliding Window, ask: "Is this question on an Array or a String?"

-  **Works on:** Arrays and Strings (a string is just an array of characters, so it counts).
-  **Does NOT work on:** Linked Lists — in 99.9% of cases. Tattoo this rule in your memory.

Checkpoint #2 — Contiguous, Not Random

The second filter: the question must be about a SUBARRAY (for arrays) or SUBSTRING (for strings) — meaning the elements must be CONTIGUOUS (next to each other, no skipping). It must NOT be a subsequence problem.

Term	Contiguous?	Meaning	Example from [1, 0, 3, 5]
Subarray	✓ Yes	Continuous block of an array	[0, 3, 5] is valid, [1, 3, 5] is NOT
Substring	✓ Yes	Continuous block of a string	"abc" of "abcd" is valid, "ac" is NOT
Subsequence	✗ No	Any elements, order preserved, gaps allowed	[1, 3] or [1, 5] are both valid

Common Confusion

A LOT of people see "subsequence" in a question and still try to force Sliding Window onto it.

Rule of thumb: Subarray / Substring = contiguous = Sliding Window is allowed.

Subsequence = non-contiguous = Sliding Window does NOT apply — look at DP / recursion instead.

3. The Identification Flowchart (3-Question Test)

Once a question passes Checkpoint #1 and #2 above, run it through one final filter — what is the question actually asking you to find?

Keyword Checklist

If the question uses ANY of these words, Sliding Window is almost certainly the right pattern:

Category	Keywords to look for
Optimization	Maximum, Minimum, Longest, Shortest
Aggregation	Sum, Count, Average
Constraint-based	At Most K, At Least K, Exactly K

The Full 3-Step Flowchart

1. **Is it an Array or String question?** → If NO, stop. Sliding Window does not apply.
2. **Does it involve a Subarray / Substring (contiguous)?** → If NO (it's a subsequence), stop.
3. **Does it ask for Max / Min / Longest / Shortest / Sum / Count / Average / At Most K / At Least K / Exactly K?** → If YES, use Sliding Window!

Golden Rule

Array or String + Subarray or Substring (contiguous) + one of the keywords above
= Sliding Window, every single time.

4. Two Flavors of Sliding Window

Sliding Window comes in two forms, and picking the right one depends on a single clue in the question: does it give you a fixed length/size?

1 Fixed-Size Window

- Used when the problem explicitly gives you a window length (e.g. "subarray of size K", "window of length 3").
- The window size NEVER changes — both low and high move forward together, one step at a time.

2 Dynamic-Size Window

- Used when NO fixed length is given — the window size is not known in advance.
- The window expands (high++) and shrinks (low++) independently based on a condition, growing and shrinking as needed.
- This is the more advanced variant — it gets its own dedicated patterns/videos, but the core engine is the same low/high logic explained above.

Quick Test

Question gives you a size/length number (like $K = 3$)? → Fixed Window.

Question does NOT give a size, and instead asks for the "longest" / "smallest" valid window? → Dynamic Window.

5. How a Fixed Window Slides — The Core Trick

This is the single most important idea in the entire pattern — and the reason Sliding Window beats brute force.

✗ The Naive (Brute Force) Way

For every new window position, re-add up every single element inside it from scratch.

This costs $O(K)$ work per window, and there are about $O(N)$ windows $\rightarrow O(N \times K)$ total. Slow.

✓ The Sliding Window Way

Don't recompute. REUSE the previous window's answer and just adjust for what changed:

The One Formula to Remember

```
new_window_sum = old_window_sum
                 - arr[low]      // the element that just LEFT the window
                 + arr[high]     // the element that just ENTERED the window
```

Because only ONE element leaves and ONE element enters every time the window slides by one step, each slide costs just $O(1)$ instead of $O(K)$. Across the whole array that brings total time down from $O(N \times K)$ to $O(N)$.

6. Worked Example — Maximum Sum Subarray of Size K

🔗 Problem Statement

Given an array, find the maximum sum among all contiguous subarrays of a fixed size K.

Array: [1, 0, 3, 5, 2, 1, 4] K = 3

6.1 Brute Force Approach

Generate every possible subarray of size K, sum each one individually, and track the maximum.

C++ — Brute Force | Time: $O(N \times K)$ | Space: $O(1)$

```
#include <bits/stdc++.h>
using namespace std;

int maxSumBruteForce(vector<int>& arr, int k) {
    int n = arr.size();
    int maxSum = INT_MIN;

    // Try every starting index of a window of size k
    for (int i = 0; i <= n - k; i++) {
        int currentSum = 0;
        for (int j = i; j < i + k; j++) {    // re-sum every time
            currentSum += arr[j];
        }
        maxSum = max(maxSum, currentSum);
    }
    return maxSum;
}
```

⚠️ Why This Is Slow

The inner loop re-adds elements that were already added in the previous window.

Example: window [1,0,3] and window [0,3,5] both re-add 0 and 3 — that work is wasted.

6.2 Sliding Window Walkthrough (Step-by-Step)

Array: [1, 0, 3, 5, 2, 1, 4] with indices 0, 1, 2, 3, 4, 5, 6. $K = 3$.

Step 1 — build the FIRST window normally (this is the only window we sum the slow way):

```
Window 1 = arr[0..2] = [1, 0, 3]
sum = 1 + 0 + 3 = 4
```

From here on, every next window REUSES the previous sum:

Win#	Elements	Element Leaving (-)	Element Entering (+)	Calculation	Sum
1	[1, 0, 3]	—	—	$1 + 0 + 3$	4
2	[0, 3, 5]	1	5	$4 - 1 + 5$	8
3	[3, 5, 2]	0	2	$8 - 0 + 2$	10
4	[5, 2, 1]	3	1	$10 - 3 + 1$	8
5	[2, 1, 4]	5	4	$8 - 5 + 4$	7

Result

Sums obtained: 4, 8, 10, 8, 7

Maximum Sum = 10 (from window [3, 5, 2])

6.3 Optimized Sliding Window Approach

C++ — Optimized Sliding Window | Time: $O(N)$ | Space: $O(1)$

```
#include <bits/stdc++.h>
using namespace std;

int maxSumSlidingWindow(vector<int>& arr, int k) {
    int n = arr.size();
    if (n < k) return -1;          // edge case: not enough elements

    int windowSum = 0;
    // Step 1: build the very first window (indices 0 to k-1)
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    // Step 2: slide the window across the rest of the array
    for (int high = k; high < n; high++) {
        int low = high - k;        // index of the element leaving
        windowSum += arr[high];    // add the new element entering
        windowSum -= arr[low];     // remove the old element leaving
        maxSum = max(maxSum, windowSum);
    }

    return maxSum;
}

int main() {
    vector<int> arr = {1, 0, 3, 5, 2, 1, 4};
    int k = 3;
    cout << "Maximum sum of subarray of size " << k
         << " = " << maxSumSlidingWindow(arr, k) << endl;
    return 0;
}
```

Output:

```
Maximum sum of subarray of size 3 = 10
```

7. Fixed Window vs Dynamic Window — Side by Side

Aspect	Fixed-Size Window	Dynamic-Size Window
Window size	Given in the question (e.g. $K = 3$)	Not given — changes as we go
low movement	Moves every step, together with high	Moves only when a condition is broken
high movement	Moves every step, together with low	Moves every step (always expands first)
When to use	Question explicitly states a length/size	Question asks for longest / shortest / smallest valid window
Typical asks	"Subarray of size K "	"Longest substring with...", "Smallest subarray with $\text{sum} \geq X$ "
Example problems	Max sum subarray of size K , first negative in window K	Longest substring without repeating characters, min window substring
Time complexity	$O(N)$	$O(N)$ (amortized — each pointer moves at most N times)

8. Common Mistakes & Pro Tips

🚫 Mistakes to Avoid

- **Forcing it on subsequence problems:** if order can skip elements, it's NOT Sliding Window territory.
- **Forcing it on Linked Lists:** Sliding Window needs $O(1)$ random access (`arr[i]`) to add/remove elements at both ends instantly — linked lists don't give you that.
- **Moving only one pointer in a fixed window:** for a FIXED size window, low and high must move together, every step. Moving only high changes the window size, which breaks the "fixed" assumption.
- **Re-summing every window:** if you find yourself looping through the whole window again at every step, you have accidentally fallen back to brute force — go back to the subtract-old/add-new trick.
- **Ignoring the $n < k$ edge case:** always check if the array is even big enough to hold one window of size K before starting.

💡 Pro Tips

- Before coding, physically draw the array on paper, mark low and high with arrows, and dry-run 2-3 slides by hand.
- Whenever you see "contiguous", "subarray", or "substring" in a problem statement, your brain should immediately whisper "Sliding Window?".
- A neat trick for "exactly K " distinct-element problems: **`exactly(K) = atMost(K) - atMost(K - 1)`**. Solve the easier "at most" version twice instead of the harder "exactly" version once.
- Always state out loud which type you're using — fixed or dynamic — before writing a single line of code. It decides your whole loop structure.

9. Complexity Cheat-Sheet

Approach	Time Complexity	Space Complexity	Notes
Brute Force (any window problem)	$O(N \times K)$	$O(1)$	Re-sums every window from scratch
Sliding Window — Fixed Size	$O(N)$	$O(1)$	Each element added once, removed once
Sliding Window — Dynamic Size	$O(N)$	$O(1)$ or $O(K)$ for a hashmap/set	Amortized: low + high each move $\leq N$ times total

10. Practice Problems (Build Muscle Memory)

Fixed-Size Window

4. Maximum / Minimum sum of a subarray of size K
5. First negative number in every window of size K
6. Maximum of all subarrays of size K (sliding window maximum — deque-based)
7. Count occurrences of anagrams of a pattern in a string
8. Average of all contiguous subarrays of size K

Dynamic-Size Window (preview — covered in the next pattern videos)

9. Longest substring without repeating characters
10. Smallest subarray with a sum greater than or equal to a target
11. Longest subarray with sum less than or equal to K
12. Minimum window substring
13. Longest substring with at most K distinct characters

11. Interview Q&A

Q: What is the relationship between Two Pointers and Sliding Window?

A: Sliding Window is built directly on top of Two Pointers. Two Pointers gives you two boundary indices moving through a structure; Sliding Window adds the specific rule that those two pointers define a contiguous "window" that can expand (move high) or shrink (move low), used to track a running aggregate like a sum or count.

Q: Why doesn't Sliding Window work on Linked Lists?

A: Sliding Window's efficiency comes from $O(1)$ access to `arr[low]` and `arr[high]` to instantly add or remove an element from either end of the window. Linked lists don't support $O(1)$ random access — finding a specific index requires walking from the head, which would defeat the purpose of the optimization.

Q: What's the difference between a subarray and a subsequence?

A: A subarray (or substring, for strings) must be contiguous — no skipping elements. A subsequence preserves relative order but can skip elements freely. Sliding Window only ever applies to subarray/substring problems, never to subsequence problems.

Q: How do you decide between a fixed-size and a dynamic-size window?

A: Check if the problem statement hands you an explicit length (like "size K" or "window of length 3"). If yes, use a fixed window where both pointers move together. If the problem instead asks for the longest/shortest/smallest valid window without giving a size, use a dynamic window where the pointers move independently based on a condition.

Q: What time complexity improvement does Sliding Window give for the fixed-size subarray sum problem?

A: Brute force recomputes every window's sum from scratch: $O(K)$ work for each of roughly N windows, giving $O(N \times K)$ overall. Sliding Window reuses the previous window's sum and only adjusts by the one element leaving and the one element entering, giving $O(1)$ work per slide and $O(N)$ overall.

Q: Walk through how the window sum is updated when the window slides by one position.

A: $\text{newSum} = \text{oldSum} - \text{arr}[\text{low}] + \text{arr}[\text{high}]$, where `arr[low]` is the element that just exited the window on the left and `arr[high]` is the new element that just entered on the right. After updating the sum, both `low` and `high` are incremented (for a fixed-size window) to move the window one step forward.

Q: Can Sliding Window be used for "exactly K distinct elements" type problems?

A: Yes, indirectly. "Exactly K" is awkward to track directly, but it can be expressed as $\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K - 1)$. Solve the simpler "at most K" version (a standard dynamic window problem) twice with different K values and subtract.

Q: What is the space complexity of the Sliding Window technique?

A: For simple sum/count tracking, $O(1)$ extra space, since you only maintain a couple of running variables (sum, low, high). If the window needs to track frequency of elements (e.g. distinct character counts), it becomes $O(K)$ or $O(\text{alphabet size})$ for the hashmap/array used to store frequencies.

12. One-Page Quick Revision

Sliding Window — Cheat Sheet

1. Applies to: Array or String only (never Linked List).
2. Must be: Subarray / Substring (contiguous) — never Subsequence.
3. Keywords: Max, Min, Longest, Shortest, Sum, Count, Average, At Most K, At Least K, Exactly K.
4. Two types: FIXED size (size given, both pointers move together) vs DYNAMIC size (size not given, pointers move independently).
5. Core trick: $\text{newSum} = \text{oldSum} - \text{arr}[\text{low}] + \text{arr}[\text{high}] \rightarrow O(1)$ per slide instead of $O(K)$.
6. Total complexity: $O(N \times K)$ brute force $\rightarrow O(N)$ sliding window.

Next up: Dynamic-Size Sliding Window — where low and high move independently, and each problem has its own "shrink condition". Practice the fixed-window problems above first to build the base reflex before moving on.